

IMC Prosperity

Complete Study Guide — Rounds 1 through 3 (2023–2025)

A comprehensive analysis of all manual & algorithmic challenges for Prosperity 4 preparation

How to Use This Guide

This guide covers every round of IMC Prosperity 1 (2023), Prosperity 2 (2024), and Prosperity 3 (2025), synthesised from the code and writeups of top-finishing teams including Stanford Cardinal (#2, 2023), camel_case (#9, 2024), and Frankfurt Hedgehogs (#2, 2025). For each round you will find the manual challenge problem statement and solution approaches, the algorithmic challenge products and strategies, key concepts and techniques used, and targeted advice for Prosperity 4 preparation.

Competition Structure

Each Prosperity competition runs for ~15 days across 5 scored rounds (plus a tutorial round). Each round has two components: (1) an Algorithmic Trading challenge where you submit a Python `trader.py` that is evaluated against a bot marketplace, and (2) a Manual Trading challenge where you solve an optimisation or game-theory problem. Algorithmic PnL dominates the final score. All products from previous rounds carry forward, so your final round algorithm must trade everything introduced so far.

Prosperity 4 Tip

Start building your backtesting infrastructure before the competition begins. The #2 team in both Prosperity 2 and Prosperity 3 credit early tooling (visualiser + local backtester) as a major competitive advantage. Jmerle's open-source backtester works for all three past years.

IMC Prosperity 1 (2023)

Prosperity 1 introduced the competition format to the world. Roughly 7,007 teams competed. The fictional setting is an island economy that trades real goods using SeaShells as currency. Each round adds new products while keeping old ones live. The Stanford Cardinal team finished 2nd overall with a clean, well-engineered market-making and linear-regression approach.

Tutorial Round

Algorithmic Challenge

Products introduced: PEARLS and BANANAS. The tutorial familiarises participants with the Python trading API, the order-book structure, the position limit system, and how to submit algorithms.

PEARLS had an extremely stable true fair value of exactly 10,000 SeaShells. The order book always hovered tightly around this value. BANANAS were more volatile with no obvious fixed fair value.

Key concepts introduced: Order Book (best bid/ask), Position Limits (max units you can hold long or short), Market Making (posting both buy and sell orders to earn the spread), and Market Taking (hitting existing orders to trade immediately). The fundamental trading loop in Python: iterate over the order book, compute fair value, post orders around it.

What Is Market Making?

A market maker simultaneously posts a buy order (bid) slightly below fair value and a sell order (ask) slightly above it. When both fill, you earn the spread. For PEARLS at fair value 10,000 you might bid 9,998 and offer 10,002 — you earn 4 SeaShells per round-trip. The risk is holding a large one-sided position if the market keeps moving against you, so position limits matter.

Round 1

Manual Challenge

Problem: You are a fish-gear trader on the island. Fish arrive one at a time, each with a private reserve price uniformly drawn from [900, 1000]. If your posted bid is at or above a fish's reserve price, it sells you gear and you earn $(1000 - \text{your_bid})$ per piece. You must submit TWO bids: a lower bid (low) and a higher bid (high). Each fish sells to the lower bid if $\text{reserve} \leq \text{low}$, and to the higher bid if $\text{reserve} \leq \text{high}$ (but $> \text{low}$). You are guaranteed to resell all gear at 1000 at the end.

This is a classic mechanism design / revenue optimisation problem. Because the reserve prices are uniformly distributed, you can write the expected profit as a closed-form function of your two bid prices and maximise it analytically.

Solution Approaches

Approach 1 — Simulation (most common): Generate 10 million random reserve prices, loop over a grid of (low, high) bid pairs, and compute average profit. This is straightforward to code and reliably finds the optimum. Stanford Cardinal and many other teams used this.

Approach 2 — Closed-Form Calculus: The expected profit from a single bid p against a uniform[900,1000] distribution is $E[\text{profit}] = (1000 - p) * (p - 900)/100$. Taking the derivative and setting to zero gives the optimal single-bid price of 950. For two bids, you split the fish population and solve the coupled system: optimal answers are low = 952 and high = 978. Edgar Maddocks published a detailed closed-form solution writeup after Round 1 that confirms this result exactly.

Optimal answer: low bid = 952, high bid = 978. The jmerle team (9th in Prosperity 2) confirmed these numbers via 10M-draw simulation. Teams that submitted 950/980 were also close, with roughly the same ranking impact.

Prosperity 4 Tip

The manual challenge in Prosperity rounds almost always involves an optimisation under uncertainty. The key skill is converting the word problem into a mathematical expectation, then either solving it analytically or brute-force searching. Always code a simulation as a sanity-check even if you derive an analytic solution.

Algorithmic Challenge

Products active: PEARLS, BANANAS. Position limits were typically 20 per product.

PEARLS — Mean-Reversion / Market Making at 10,000: Since 10,000 is the known true price, the strategy is simply to always bid below 10,000 and offer above 10,000, and to take any orders that cross this fair value. Stanford Cardinal noticed moments when the best bid was 10,002 and the best ask was 9,998 — an immediate profit opportunity. The implementation: check if any sell order is at or below 10,000 (buy it), check if any buy order is at or above 10,000 (sell to it), and post passive limit orders around 10,000 to collect the spread.

Popular Price (Wall Mid)

A key insight from the #2 team: instead of using the arithmetic mid-price, use the 'popular price' — the buy or sell price with the maximum volume on that side of the book. This is a more robust estimate of fair value because bot market makers tend to cluster their large orders at the true fair value. Jmerle also used this concept in Prosperity 2. In later rounds it became known as the 'Wall Mid'.

BANANAS — Price Prediction & Market Making: BANANAS had no fixed fair value but moved in predictable short-term patterns. Stanford Cardinal observed the price was driven by a moving average of recent mid-prices. Their approach: (1) compute a rolling mean of recent mid-prices as a fair value estimate, (2) make markets around this estimate. Some teams tried to regress BANANAS price on PEARLS as a covariate, but the relationship was weak. The benbates30 repo even trained a logistic regression classifier to predict price direction using lagged mid-prices and volume features as input features — a machine learning approach, though results were mixed compared to simple market making.

Common Mistake / Watch Out

One very common mistake in early rounds is forgetting to handle position limits. If your algorithm keeps buying and never sells, you hit the limit and cannot trade. Always build in a soft-stop (start reducing quotes) and hard-stop (liquidate aggressively) when approaching position limits. The Stanford Cardinal code has an explicit 'soft limit' at 18 and 'hard limit' at 20 for position management.

Round 2

Manual Challenge

Problem: An FX / barter exchange chain problem. You are given exchange rates between several goods (e.g., Pizzas, Wasabi Roots, SeaShells, Snowballs, Silicon Nuggets). Starting with 1 SeaShell, using at most N trades, find the sequence of conversions that maximises your final SeaShell balance.

This is a classic graph problem where nodes are goods and edges are exchange rates. You want the highest-weight cycle starting and ending at SeaShells within N steps. The trick is that the rates are multiplicative, so you take the product of edge weights along a path.

Solution Approaches

Approach 1 — BFS/DFS with Pruning: Build a directed graph of all possible conversions. Run a BFS or DFS from SeaShells, exploring all paths of length up to N , tracking the running product of exchange rates. Return the path with the highest final SeaShell value.

Approach 2 — Dynamic Programming: Let $dp[k][g]$ = maximum SeaShells-equivalent value achievable at step k holding good g . Iterate forward for $k = 1$ to N . The answer is max over all g of $dp[N][\text{SeaShells}]$.

Approach 3 — Brute Force Iteration (works for small N): Since N is typically 5–6, simply enumerate all possible conversion sequences of length up to N . The number of paths is at most $(\text{number of goods})^N$ which is feasible for 5–6 goods and 5 steps.

Key result: In Prosperity 2 (where this problem reappeared), the optimal chain was SeaShells → Pizza Slices → Wasabi Roots → SeaShells → Pizza Slices → SeaShells, giving 113,938.8 SeaShells from a starting balance. The jmerle team found this by iterating over all possible conversion combinations.

Prosperity 4 Tip

Whenever you see a graph of exchange rates, think immediately: is there an arbitrage cycle? If the product of rates around a cycle is greater than 1, you can profit by traversing it repeatedly. Log-transform the rates to convert products into sums, making it easier to apply standard shortest-path algorithms (Bellman-Ford finds negative cycles efficiently).

Algorithmic Challenge

Products active: PEARLS, BANANAS, + new products (exact P1 R2 products varied by source, likely COCONUTS and PINA_COLADAS based on the overall product arc). The key skill introduced here is understanding correlated products.

Correlated Pairs Strategy: COCONUTS and PINA_COLADAS had a structural relationship — Pina Coladas are made from Coconuts. If the ratio of PINA_COLADA price to COCONUT price deviates significantly from its historical mean, you trade the spread: buy the cheap side and sell the expensive side simultaneously. This is the foundational concept of Pairs Trading / Statistical Arbitrage.

Statistical Arbitrage (Stat-Arb) Primer

Two products X and Y are 'cointegrated' if the spread $S = X - \beta Y$ is stationary (mean-reverting). Strategy: (1) Estimate beta (the hedge ratio) from historical data using linear regression or OLS. (2) Compute the current spread and its z-score: $z = (S - \text{mean}(S)) / \text{std}(S)$. (3) When $z > \text{threshold}$ (say 2.0), short X and buy Y. When $z < -\text{threshold}$, long X and short Y. Exit when z returns to near 0. This earns money from mean-reversion without directional market exposure.

Round 3

Manual Challenge

Problem: A variant of the treasure map / tile selection game. Participants are presented with a grid of tiles, each with a multiplier. The reward from a tile is: $(7,500 \times \text{multiplier}) \div (1 + \text{number_of_other_hunters_on_that_tile})$. The number of hunters on a tile increases proportional to how many participants select it. You can select multiple tiles but each additional search costs money. This is a game-theoretic Nash Equilibrium problem.

Solution Approaches

Approach 1 — Nash Equilibrium Calculation: In a Nash equilibrium, all chosen tiles yield the same expected reward. You can solve for the equilibrium occupancy of each tile by setting rewards equal and solving the system of equations. If a tile's reward at zero occupancy exceeds the equilibrium reward of popular tiles, some players will rationally switch to it.

Approach 2 — Simulation with Behavioral Assumptions: Assume all other players act rationally and calculate the Nash equilibrium occupancy. Then compute your expected reward on each tile under those occupancies. Pick the tile with the highest remaining EV. The David Teather team used this and picked I26 and H29 — the 1st and 6th optimal tiles respectively under the rational-play assumption.

Approach 3 — Contrarian Betting: Assume many players will crowd the highest-multiplier tiles without thinking carefully about competition. Deliberately pick slightly lower multiplier tiles that are likely to be less crowded. This hedge strategy consistently performs well across all Prosperity iterations.

Prosperity 4 Tip

In all treasure/suitcase/container manual problems across all three Prosperity competitions, the pattern is the same: raw multiplier is misleading, expected value after dilution from crowding is what matters. Always model expected occupancy before picking. The crowded 'obvious' tiles rarely win.

Algorithmic Challenge

Products active: PEARLS, BANANAS, COCONUTS, PINA_COLADAS. The major addition in this round is the cointegrated pair COCONUTS/PINA_COLADAS. The key insight: because one Pina Colada requires Coconuts to make, their prices have a forced structural relationship.

Pairs Trading on COCONUTS/PINA_COLADAS: Compute a rolling Z-score of the spread between PINA_COLADAS and (coefficient $\times$ COCONUTS). Trade when $|Z| > 1.5$ –2.0: if Z is high, sell PINA_COLADAS and buy COCONUTS; if Z is low, buy PINA_COLADAS and sell COCONUTS. This strategy requires careful hedge ratio calibration — teams used both simple ratios (e.g. 1 Pina Colada = 2 Coconuts) and OLS regression.

Position Sizing: Because you are trading both legs simultaneously, you must coordinate position limits across both products. When you are at the limit on COCONUTS, you cannot add to the long side of the pair trade even if the Z-score demands it. Top teams built explicit pair-aware position management.

Round 4

Manual Challenge

Problem: Gear-from-fish problem again, but with a twist. The first bid works the same as Round 1 (standard uniform reserve price). The second bid, however, is evaluated relative to the average of all participants' second bids: if your bid is above the average, you sell at full profit; if your bid is below the average, your profit on those trades is scaled down by a cubic penalty function. This introduces a game-theoretic / Keynesian beauty contest dimension.

Solution Approaches

The optimal first bid is still ~952, derived from the same uniform distribution logic as Round 1. For the second bid, the penalty for being below average creates pressure to bid higher than your naive optimum. The rational equilibrium shifts from 978 upward. The David Teather team picked 960/980, reasoning that the second-bid equilibrium would shift slightly upward from the Round 1 values due to the average penalty.

Common Mistake / Watch Out

This is a classic 'beauty contest' problem: the optimal answer depends on what you expect others to submit, which depends on what they expect you to submit, and so on. Pure game-theory analysis gives a fixed point, but in practice participants' bids cluster around 'near-optimal' values. The safest approach: use the analytically optimal bid for the first slot, and submit slightly above average-expectation for the second slot to avoid the cubic penalty.

Algorithmic Challenge

Products active: PEARLS, BANANAS, COCONUTS, PINA_COLADAS, BERRIES, DIVING_GEAR. BERRIES showed strong time-of-day seasonality — prices peaked at certain timestamps and bottomed at others within each day. DIVING_GEAR prices were correlated with DOLPHIN_SIGHTINGS, an external signal stream.

BERRIES — Seasonal / Time-Series Strategy: Plot the mid-price of BERRIES against the timestamp index. You will see a clear pattern: prices rise for the first half of the day and fall in the second half (or vice versa). Strategy: buy at the known seasonal trough, hold, and sell at the known seasonal peak. This is a pure calendar-effect trade. Position management is critical: you must build the position gradually to avoid moving the market, and you must exit before the price reversal.

DIVING_GEAR / DOLPHIN_SIGHTINGS — External Signal Strategy: The DOLPHIN_SIGHTINGS stream acts as a leading indicator for DIVING_GEAR prices. When sightings spike, diving demand (and thus gear prices) rises with a lag. Strategy: monitor the sightings stream, and when it changes significantly, enter or exit DIVING_GEAR positions accordingly. This introduced the concept of trading based on an external fundamental signal rather than just the price series.

Prosperity 4 Tip

The BERRIES seasonal pattern and the DOLPHIN_SIGHTINGS correlation are deliberately engineered signals. In Prosperity competitions, if you plot price or an external variable against time and see a clear pattern, trade it aggressively — these signals are usually not coincidental. Always visualise your data before building an algorithm.

Round 5

Algorithmic Challenge

The final round typically introduces the last batch of products and requires teams to run all previous strategies simultaneously. In Prosperity 1 Round 5, all previously introduced products were active and the main challenge was to maintain well-calibrated strategies across the entire portfolio while managing correlated risks. Top teams spent this round debugging and refining parameters rather than introducing new strategies, as the risk of breaking existing profitable strategies outweighed the upside of new additions.

Key Prosperity 1 Algorithmic Concepts Summary

Market Making (PEARLS/BANANAS): Post orders around fair value, earn the spread. Fair value estimation: use the 'popular price' (volume-weighted mode of the book) rather than plain mid. Position management: implement soft and hard limits to avoid being stuck at the position boundary. Pairs Trading (COCONUTS/PINA_COLADAS): compute spread Z-score, trade mean-reversion. Seasonality (BERRIES): plot price vs time, trade the cycle. External signal (DIVING_GEAR): use DOLPHIN_SIGHTINGS as a leading indicator.

IMC Prosperity 2 (2024)

Prosperity 2 ran in April–May 2024 with ~9,000+ teams. It substantially raised the difficulty, introducing Orchids (a product with complex conversion mechanics mimicking real options-adjacent behaviour), a gift basket decomposition challenge (ETF arbitrage), and Coconut Coupons (an options product). The jmerle team finished 9th globally, 3rd in Europe, and 1st in the Netherlands.

Tutorial Round

Algorithmic Challenge

Products: AMETHYSTS and STARFRUIT. AMETHYSTS were Prosperity 2's version of PEARLS — a fixed fair value at 10,000. STARFRUIT was a more volatile product analogous to BANANAS.

AMETHYSTS Strategy: Exactly the same as PEARLS in Prosperity 1. Bid below 10,000, offer above 10,000, and aggressively take any orders that cross fair value. Many participants simply reused the P1 PEARLS code with the product name changed.

STARFRUIT Strategy: The jmerle team used the 'popular buy and sell price' (volume-weighted dominant price on each side of the book) to estimate fair value. Starfruit's fair value was taken as the midpoint between the popular buy and sell prices. A linear regression on recent mid-prices also worked for prediction. The benbates30 ML approach (logistic regression on lagged features) also appeared in Prosperity 1 code and is worth noting: use recent mid-prices at various lags as features, predict up/down, size position accordingly.

Linear Regression for Price Prediction

For an asset like STARFRUIT, fit a model: $\text{mid_price}[t] = a_0 + a_1 \cdot \text{mid}[t-1] + a_2 \cdot \text{mid}[t-2] + \dots + \text{epsilon}$. If the predicted mid exceeds the current market mid, buy; if below, sell. This is an AR(p) model — an autoregressive price predictor. In practice in Prosperity 2, teams found this added meaningful alpha over pure market making for STARFRUIT. The key is to calibrate the lookback window (typically 4–6 lags work well).

Round 1

Manual Challenge

Problem: Identical to Prosperity 1 Round 1 — the secretive fish gear problem with two bids and a uniform reserve price distribution on [900, 1000]. This round established that Prosperity 2 would revisit and remix ideas from Prosperity 1, so having studied last year was an immediate advantage.

Solution is identical: low bid = 952, high bid = 978. The jmerle team ran a simulation with 10 million random reserve prices and a grid search over bid pairs, arriving at the same result. The jmerle writeup confirmed these numbers matched Edgar Maddocks' closed-form solution exactly.

Algorithmic Challenge

Products: AMETHYSTS, STARFRUIT. This was effectively a continuation of the tutorial. Most teams submitted near-identical code to their tutorial submission. The primary differentiator was quality of fair value estimation for STARFRUIT. The jmerle team found no improvement over their tutorial algorithm and submitted without changes.

Key takeaway: In the early rounds, the market-making quality for the 'stable' product (AMETHYSTS) is nearly identical across all serious competitors. The edge comes from the 'unstable' products like STARFRUIT. Spending time on fair value estimation and position management for the noisy product pays off more than micro-optimising the stable one.

Round 2

Manual Challenge

Problem: FX/barter conversion chain. Starting with SeaShells, use at most a certain number of trades to convert through a sequence of goods (Pizza Slices, Wasabi Roots, Snowballs, etc.) and back to SeaShells, maximising the final balance. The goods and rates are given explicitly.

Solution: The jmerle team enumerated all possible conversion sequences and computed the product of exchange rates for each path. The optimal path was SeaShells → Pizza Slices → Wasabi Roots → SeaShells → Pizza Slices → SeaShells, yielding a 113,938.8 SeaShell profit — a near 11.4% gain in a single round. This is essentially the same problem as Prosperity 1 Round 2.

Prosperity 4 Tip

Always have a general graph-search tool ready for these problems. A generic BFS that explores all N-step paths from node 'SeaShells' back to 'SeaShells' takes about 5 minutes to code and solves all FX chain problems regardless of the specific goods/rates given each year. The key is to represent the exchange rates as a weighted directed graph and search for the maximum-product cycle.

Algorithmic Challenge

New product: ORCHIDS. This was the hardest product in Prosperity 2 for most teams. Orchids had a complex conversion mechanism: you could convert between holding orchids locally versus importing/exporting them through an external 'South' marketplace that had its own

bid/ask prices, transport fees, import tariffs, and export tariffs. The orchid's local market price fluctuated based on factors including sunlight intensity, humidity, and a shipping schedule.

Understanding the Orchid Conversion Mechanics: The key insight — which most teams missed initially — is that conversion allows you to go short ORCHIDS: you sell ORCHIDS locally and then immediately convert your short position to zero via the South marketplace. The profitable strategy was: (1) Short-sell ORCHIDS at the local best bid, (2) In the same iteration, convert your short position back to flat by importing from the South at a price that, net of all fees, is lower than what you sold locally for. If $(\text{local bid}) - (\text{south ask} + \text{fees}) > 0$, you profit on each round-trip.

The 60k+ Strategy (jmerle hint): The trick was to continuously short-sell to the position limit at a price at which you could immediately convert profitably back to 0 in the next iteration. After getting a hint on Discord, jmerle refined this further: you can convert your position to 0 AND immediately re-enter the short position in the same iteration, essentially running the conversion arbitrage at maximum throughput. This jumped PnL from a few thousand to 60k+, then 109k after the same-iteration refinement.

Conversion Arbitrage Pattern

This pattern — selling/buying locally and using a conversion/import/export mechanism to close the position — recurs in later Prosperity rounds (e.g. Magnificent Macarons in P3 R4). The general structure: Effective import cost = South Ask + transport fee + import tariff. Effective export revenue = South Bid - transport fee - export tariff. If local ask < effective import cost: buy locally, import to close. If local bid > effective export revenue: sell locally, export to close. The basis between local and conversion prices is your profit per unit.

Common Mistake / Watch Out

The Orchid round was where many teams diverged sharply in performance. Teams that figured out the conversion trick earned 60k–109k in a single round; teams that missed it earned almost nothing on ORCHIDS. The lesson: always read the product description extremely carefully and think about all the mechanisms available to you, not just the standard order-book trading.

Round 3

Manual Challenge

Problem: Treasure map tile hunt — the same game-theory problem from Prosperity 1 Round 3, with a different grid. Participants see a map with tiles of different multipliers and known default occupancy. Each tile's reward is split by the total number of hunters on it. Multiple searches are available: the first is free, the second costs 25,000, and the third costs 75,000 SeaShells.

The David Teather team assumed rational play, computed EV for each tile, and selected the 1st and 6th optimal tiles as their two picks (I26, H29) — hedging against crowds going to the pure

maximum. Most teams found that picking a high but not highest multiplier tile with low default occupancy gave the best risk-adjusted EV.

Algorithmic Challenge

New products: CHOCOLATE, STRAWBERRIES, ROSES, and GIFT_BASKET. The GIFT_BASKET contained exactly 4 CHOCOLATE + 6 STRAWBERRIES + 1 ROSES. You could not physically assemble or disassemble baskets — you could only trade the individual components and the basket independently. This is an ETF arbitrage setup.

ETF Arbitrage on Gift Basket: Compute a synthetic fair price for the basket: $\text{Synthetic} = 4 \times \text{CHOCOLATE_mid} + 6 \times \text{STRAWBERRIES_mid} + 1 \times \text{ROSES_mid}$. Compare this to the actual GIFT_BASKET mid. If $\text{Basket} > \text{Synthetic} + \text{threshold}$: the basket is rich — sell the basket, buy the components. If $\text{Basket} < \text{Synthetic} - \text{threshold}$: the basket is cheap — buy the basket, sell the components. The threshold must cover your trading costs and slippage. This is statistical arbitrage applied to a known structural composition.

Z-Score Approach: Rather than a fixed threshold, compute a rolling Z-score of $(\text{Basket} - \text{Synthetic})$ and trade when $|Z| > 1.5\text{--}2.0$. This adjusts to changing volatility regimes and is more robust. Exit when Z returns toward 0. The David Teather team used a similar 'weighted sum of individual items' to price the basket and found this round particularly profitable.

Component-Level Market Making: Simultaneously run market making on each of CHOCOLATE, STRAWBERRIES, and ROSES individually, using their own price series. The basket arbitrage is an additional overlay on top of these. The jmerle code shows this clearly: Round 3 was 'basically a copy-paste of last year's Round 4' (COCONUTS/PINA_COLADAS pairs trading), applied to the new basket structure.

Prosperity 4 Tip

ETF arbitrage is one of the most reliable strategies in all quant trading. In Prosperity, whenever you see a product that is defined as a fixed linear combination of other products, immediately set up the basis trade. The key variables to calibrate are: (1) the threshold for entry (how many SeaShells of mispricing justifies trading), (2) position limits on each leg, and (3) whether to hedge delta-neutral (match component quantities exactly) or just trade one side.

Round 4

Manual Challenge

Problem: The gear-from-fish problem returns with a third twist. The second bid is now benchmarked against the average of all participants' second bids: if your bid exceeds the average, fish sell normally; if below the average, a penalty is applied. This is a mix of the Round 1 optimisation problem and a coordination game.

The David Teather team picked 960/980 — intending to get full first-bid profit at 960 (capturing the lower reserve-price block), while the 980 second bid was expected to be above the average (avoiding the penalty). This is consistent with the game theory: the penalty for being below average pushes everyone upward, creating an equilibrium above 978.

Algorithmic Challenge

New products: COCONUT and COCONUT_COUPON. The COCONUT_COUPON was analogous to a European call option on COCONUT with a fixed expiry at the end of the round. This introduced options pricing concepts — specifically Black-Scholes — to Prosperity.

Black-Scholes Options Pricing: The fair value of a European call option is determined by 5 inputs: current asset price (S), strike price (K), time to expiry (T), risk-free rate (r), and implied volatility (sigma). Black-Scholes gives: $C = S * N(d1) - K * e^{(-rT)} * N(d2)$, where $d1 = [\ln(S/K) + (r + \sigma^2/2)*T] / (\sigma*\sqrt{T})$ and $d2 = d1 - \sigma*\sqrt{T}$, and N() is the standard normal CDF.

Strategy: Estimate the implied volatility from historical COCONUT prices using rolling standard deviation. Feed this into Black-Scholes to get a model price for the coupon. Compare to the market price of COCONUT_COUPON: if model > market, buy the coupon; if model < market, sell it. The David Teather team built a Black-Scholes model but found a bug that unexpectedly made money when the coupon price was falling — a reminder that bugs can sometimes accidentally create profitable positions, but intentional correctness is always better.

Delta Hedging: To isolate the volatility signal and avoid directional coconut exposure, hedge the option position with the underlying: $\text{delta_hedge_size} = \text{option_delta} \times \text{coupon_position}$. Buy or sell COCONUTS equal to minus your net delta to stay delta-neutral. This allows you to trade pure implied volatility rather than direction.

Understanding Delta Hedging Intuitively

If you are long 1 call with delta = 0.5, you are effectively long 0.5 units of the underlying for every small price move. To neutralise this, sell 0.5 units of COCONUT. Now small moves in COCONUT do not affect your P&L — only changes in implied volatility do. You rebalance (re-hedge) periodically as delta changes with price and time. This process is called dynamic delta hedging and it is the foundation of options market making.

Round 5

Algorithmic Challenge

Round 5 introduced access to historical trade data including trader IDs — you could see which specific bot counterparties had executed which trades. The insight was that some bots had predictable behaviour: a bot that consistently buys before prices rise is a 'lead' bot that you can front-run. A bot that sells before drops is similarly exploitable.

Trader ID Strategy: Collect historical data mapping each trader ID to their subsequent price impact. If trader ID 'Vladimir' buys STARFRUIT and the price reliably moves up in the next 3 ticks, then when you see Vladimir buying, you buy ahead of the expected move. This is information-based alpha. The jmerle team accumulated a significant edge from this in Round 5, jumping from 9th to a comfortable finish with 369k in the final round alone.

Prosperity 4 Tip

Trader ID analysis is a recurring Prosperity mechanic. In Prosperity 3, it was also explicitly introduced in Round 5. Build a framework early that logs all trades with counterparty IDs, computes per-ID price impact over the next N ticks, and flags high-signal bots. This is a powerful alpha source that is easy to overlook if you are only focused on the price series itself.

IMC Prosperity 3 (2025)

Prosperity 3 ran in 2025 with 12,000+ teams — by far the largest field yet. The Frankfurt Hedgehogs team (Timo Diehm, Arne Witt, Marvin Schuster) finished 2nd with 1,433,876 SeaShells. Martin Oravec finished 73rd solo (5th in the UK) with no prior quant experience. The competition introduced FX arbitrage as a manual challenge, container/suitcase EV problems, options with volatility smile, location arbitrage, and a news-based portfolio round.

Tutorial Round

Algorithmic Challenge

Products introduced: RAINFOREST_RESIN (stable, like PEARLS/AMETHYSTS) and KELP (unstable). The tutorial was deliberately designed to be straightforward, establishing the basic market-making loop that all later strategies build on.

RAINFOREST_RESIN: Stable product with a known true value. Pure market making: post bids and offers symmetrically around fair value, harvest the spread. The Frankfurt Hedgehogs used a concept called the 'Wall Mid' — the price level at which the largest standing limit order sits on each side of the book. This is a much more robust fair value estimate than arithmetic mid-price because it reflects the price at which the largest market participant is willing to transact.

KELP: Unstable with no mean-reversion tendency. Martin Oravec settled on a modified market-making strategy using the current mid-price as fair value, combined with opportunistic liquidity provision: when the spread is wide, post passive quotes; when there is an obvious directional bias in the order book (large imbalance between bid and ask depth), lean your quotes in the favourable direction.

Round 1 — Market Making

Manual Challenge — FX Arbitrage

Problem: You are given a 4×4 exchange rate matrix between 4 currencies/goods. Using at most 5 trades (conversions), starting from 1 unit, maximise the final amount. This is the FX arbitrage / graph problem from earlier years, but framed as a currency exchange rather than barter.

Solution: Martin Oravec treated it as a graph problem and used a brute-force BFS to find the maximum-product path of up to 5 steps. The optimal profit was about 8.9%. Most teams found the same answer since the problem was deterministic — there is only one correct answer and it can be found reliably with brute force.

Frankfurt Hedgehogs approach: Treat as a directed weighted graph. BFS/DFS over all paths up to length 5. Return the maximum product path from SeaShells back to SeaShells. A Python implementation takes under 5 minutes to write and is perfectly sufficient.

Algorithmic Challenge

Products active: RAINFOREST_RESIN, KELP, + new product SQUID_INK.

SQUID_INK — The Polarising Product: SQUID_INK was highly contentious on Discord. Most participants could not find a reliable profitable strategy and it lost money for many who attempted to trade it. Martin Oravec tried z-score mean reversion, statistical arbitrage, and volatility recoil — all worked decently in backtests but lost money in live evaluation. He eventually chose not to trade it. However, some teams did find profitable approaches post-competition.

What worked for SQUID_INK (discovered post-competition): The product appeared to have short-term momentum followed by sharp mean-reversion — a volatility spike/recoil pattern. The correct approach was to detect large price spikes (e.g., using a short Z-score), fade the spike immediately (trade against the extreme move), and close out quickly as the price reverts. The key was using very tight holding periods and not relying on multi-period mean reversion.

The Frankfurt Hedgehogs' Wall Mid in Practice

For RAINFOREST_RESIN, they observed that the order book always had a large 'wall' of orders at specific price levels (e.g., 10000 bid / 10001 ask). These walls are placed by market-making bots. Using the midpoint of the wall prices as fair value (instead of the best bid/ask mid) gave a cleaner signal because the wall doesn't move on small noise. If the wall is at 10000/10001, fair value is 10000.5 — post your own orders inside the wall to be first in the queue.

Round 2 — ETF Statistical Arbitrage

Manual Challenge — Containers

Problem: There are 10 containers, each with a multiplier and a number of inhabitants already inside. The total pot per container = $10,000 \times \text{multiplier}$. Your share = $\text{pot} \div (\text{inhabitants} + \text{your_occupancy_contribution})$. Your occupancy contribution increases for each percentage of participants that choose the same container. The first container is free; the second costs 50,000 SeaShells.

Solution: Martin Oravec ranked containers by expected value accounting for crowding. His analysis found Container 4 ($31 \times$ multiplier, 2 inhabitants) gave the best risk-adjusted EV as a free pick: at 4–8% participant share, the gross yield is 31,000–51,700 SeaShells. He skipped the paid second pick because no container reliably cleared the 50,000 fee under realistic crowding assumptions.

This is a classic EV calculation with game-theoretic crowding: the formula is $\text{EV} = (10,000 \times \text{multiplier}) / (\text{inhabitants} + \text{expected_crowding})$. The key is modelling expected crowding accurately. The naive mistake is to always pick the highest multiplier container, ignoring that it will be the most crowded.

Prosperity 4 Tip

Across all three Prosperity competitions, the container/suitcase/tile manual problems share the same structure. Build a general EV calculator: for each option, estimate expected competitor share (usually proportional to multiplier), compute your expected payoff, and pick the highest EV option — not the highest multiplier option. A 31× container with 2 inhabitants often beats a 80× container with 15 inhabitants.

Algorithmic Challenge

New products: CROISSANTS, JAMS, DJEMBES, PICNIC_BASKET_1, PICNIC_BASKET_2. Basket 1 = 6 CROISSANTS + 3 JAMS + 1 DJEMBE. Basket 2 = 4 CROISSANTS + 2 JAMS. This is a deliberate ETF arbitrage setup — a direct callback to the Prosperity 2 Gift Basket round.

Two-Way Statistical Arbitrage: Martin Oravec built synthetic fair prices for both baskets from their component mid-prices, compared to the live basket mids, and traded the spread using a Z-score signal. He also ran the arbitrage in reverse: pricing the components off the basket prices. This two-way stat-arb creates a tighter link between basket and component prices and generates more trading opportunities.

Implementation: Synthetic price of Basket 1 = $6 \times \text{CROISSANT_mid} + 3 \times \text{JAM_mid} + 1 \times \text{DJEMBE_mid}$. Spread = Basket1_mid - Synthetic1. Z-score = $(\text{Spread} - \text{rolling_mean}) / \text{rolling_std}$. If $Z > +2$: sell Basket 1, buy components in proportion (6 croissants, 3 jams, 1 djembe). If $Z < -2$: buy Basket 1, sell components. Repeat symmetrically for Basket 2. Unwind when Z returns to 0. The Frankfurt Hedgehogs also used significant hardcoding of bot order patterns in these early rounds to 'snipe' known bot quotes.

Common Mistake / Watch Out

ETF arbitrage requires coordinating orders on multiple products simultaneously. In Prosperity's sequential order-book model, you cannot guarantee both legs fill in the same timestamp. Build your algorithm to handle partial fills: if you sell the basket but only partially fill the components, your net position is an unintended directional exposure. Track 'outstanding hedge' and prioritise filling the hedge in subsequent ticks.

Round 3 — Options and Volatility Smile

Manual Challenge — Reserve Price / Sea Turtles

Problem: A group of Sea Turtles offers flippers with reserve prices split across two ranges: [160, 200] and [250, 320]. Nothing trades in the 200–250 gap. Whatever you buy can be resold for 320. Your first bid covers the lower range; your second bid covers the upper range. On the second bid, if your bid is below the archipelago average, your profit is reduced by a cubic penalty.

Martin Oravec's analysis: For the first bid, the volume-times-spread product peaks at $p = 200$ (sweeps the low block, locks in a 120 spread per trade). For the second bid, the cubic penalty for being below average pushes rational players above the naive optimum of 285. He submitted 200 (first bid) and 290 (second bid), reasoning that the cluster would shift slightly upward but not all the way to 320.

Algorithmic Challenge

New products: VOLCANIC_ROCK and VOLCANIC_ROCK_VOUCHER (a European call option on VOLCANIC_ROCK) at several different strike prices (9,000 to 11,000 in 250-point steps).

This is the most complex product type in Prosperity history so far.

Black-Scholes Revisited and the Volatility Smile: Martin Oravec adopted the Black-Scholes framework and inverted market option prices to compute implied volatility (IV) at each strike. He observed a volatility smile: IV was higher for deep in-the-money and out-of-the-money strikes than for at-the-money strikes. This is a well-known real-world phenomenon and a classic signal for options traders.

Smile Fitting: Map each option to its log-moneyness $m = \ln(K/S)$ and fit a quadratic (parabolic) curve to (m, IV) pairs. This gives a smooth volatility surface. Price each option using its smile-fitted IV rather than a single flat volatility. Any option whose market price differs significantly from its smile-implied price is mispriced — trade it.

Delta Hedging the Options Book: Sum all option deltas across your position (long/short call vouchers at different strikes). Rebalance in VOLCANIC_ROCK whenever your net book delta moves outside a small band (e.g., ± 5 units). This keeps your P&L isolated to volatility mispricing rather than directional moves in VOLCANIC_ROCK.

Implied Volatility and the Smile — A Primer

In Black-Scholes, all options on the same underlying should have the same implied volatility (the model assumes constant vol). In reality, options at different strikes trade at different IVs because: (1) tail risks are higher than BS assumes, so far OTM options command a premium, (2) supply/demand imbalances across strikes. The 'smile' shape tells you which strikes are relatively expensive vs cheap. Trading the smile means: sell expensive IVs (high-strike calls if the right wing is elevated), buy cheap IVs (ATM options if the smile is 'smirked'), and hedge your delta to isolate the vol trade.

Frankfurt Hedgehogs' R3 approach: The #2 team used a 'scalping' strategy for options — detecting short-term mispricings between consecutive option quotes from bots and trading against them rapidly. Their dashboard showed them the vol surface in real time, allowing them to identify which strikes were offering the best risk/reward. They were more aggressive than Oravec in delta-hedging frequency and used tighter entry thresholds.

Prosperity 4 Tip

For Prosperity 4, if options reappear: (1) Compute IV for each strike using Black-Scholes inversion (binary search over sigma until BS price = market price). (2) Fit a quadratic smile curve. (3) Any option whose IV deviates > 0.5 vol points from the smile is a trading signal. (4) Always delta-hedge using the underlying. Libraries: `scipy.stats.norm` for $N(d1)$, `scipy.optimize` for IV inversion.

Round 4 — Location Arbitrage

Manual Challenge — Suitcases

Problem: 20 suitcases, same structure as the containers round. First suitcase free, second costs 50,000, third costs 100,000. Each suitcase has a multiplier and inhabitants; reward = $(10,000 \times \text{multiplier}) / (\text{inhabitants} + \text{share})$. Martin Oravec computed expected payoffs for each suitcase, ranked by EV.

His picks: C3 (73×, 4 inhabitants) as free pick (~85,900 gross EV), B4 (70×, 4 inhabitants) as paid pick (~82,400 gross, net ~32,400 after 50k fee). He skipped the third pick since no suitcase cleared the 100,000 fee. This deliberately avoided the headline 90–100× suitcases (too crowded) in favour of high-multiplier, low-attention options.

Algorithmic Challenge

New product: MAGNIFICENT_MACARONS. The trading mechanism was a location arbitrage setup: Macarons could be sourced via a conversion operator (import/export) with specific transport fees, tariffs, and a bid/ask spread, while also trading on the local open market. Additional data streams provided: sugar prices, sunlight index, import/export tariffs.

Location Arbitrage Framework: Effective import cost = $(\text{Conversion_Ask}) + \text{transport_fee} + \text{import_tariff}$. Effective export revenue = $(\text{Conversion_Bid}) - \text{transport_fee} - \text{export_tariff}$. If local ask < effective import cost: buy locally. If local bid > effective export revenue: sell locally and convert. The basis (local price - conversion price net of fees) is your P&L driver.

Sunlight Index Signal (the key insight): Martin Oravec noticed a clear regime shift in Macaron price behaviour once the sunlight index crossed a specific threshold — the import/export basis tended to flip sides at that level, suggesting a demand shift. The correct strategy: use the sunlight index as a regime indicator, switching between import-mode and export-mode based on which side of the threshold the index falls. He came close to finding this but ran out of time before fully implementing it. Teams that found and exploited this signal earned significant alpha.

Reinforcement Learning Attempt: Oravec also experimented with a reinforcement learning agent trained to decide conversion policy from features (basis, spreads, costs, sunlight). While this did not produce a deployable strategy in competition time, it is a valid approach for future rounds: use RL or supervised ML to learn regime-dependent rules from historical data.

Common Mistake / Watch Out

Macarons was the round where many teams took large losses from overfitting their strategies to the backtest data. The conversion opportunities were real but thin and short-lived. If your backtested profit on a new product is very high but your live result is poor, it is likely that (1) the backtest is overfitting to a few specific bot interactions, or (2) the signal is too intermittent to be a reliable production strategy. Test on held-out data before deploying.

Round 5 — Trader IDs and News Trading

Manual Challenge — News-Based Portfolio

Problem: You receive a newspaper with fictional news stories about several companies on the island. You must allocate 1,000,000 SeaShells across long and short positions in these companies, subject to a fee schedule that penalises very large positions in any single name (a concave fee curve, meaning over-concentrating in one name is expensive).

Martin Oravec's approach: Label each news story as strong/medium/weak buy or sell. Size positions by conviction level, subject to the fee constraint. His final portfolio included short positions in Quantum Coffee (health/regulatory overhang, 25% of portfolio), Solar Panels (tax shock, 10%), and Cacti Needle (litigation risk, 5%), with long positions in VR Monocle (user surge, 8%), Striped Shirts (fashion momentum, 5%), Ranch Sauce (takeover premium, 2%), Moonshine (launch catalyst, 5%), and Haystacks (weak conviction).

How to Approach the News Trading Round

This is a fundamental analysis problem disguised as trading. For each company, ask: (1) Is the news shock positive or negative? (2) How large and durable is the effect likely to be (is it a one-off event or a structural change)? (3) Are there any second-order effects (e.g., a competitor benefits from another's regulatory problems)? (4) How confident are you in your signal vs the fee cost? Size positions proportionally to conviction. Avoid overconcentrating — the fee curve punishes it heavily.

Algorithmic Challenge

All 15 products active. The new mechanic: you can now see trader IDs in the historical trade data, allowing you to identify bot trading patterns.

Trader ID Exploitation: Collect all trades from historical data, map each trader ID to their subsequent impact on price. Identify 'front-runnable' bots: bots that consistently trade large quantities just before a price move. When these bots appear in the live feed, position ahead of their expected impact. Frankfurt Hedgehogs called this one of the highest-alpha mechanics in the entire competition and referenced it explicitly in their README.

Portfolio Management at Scale: With 15 products live, the final round requires robust portfolio-level risk management: (1) per-product position limits, (2) global risk limit on total absolute exposure, (3) correlation awareness (being long BASKET and long its components is not

diversified), (4) ensuring strategies do not interfere with each other's order flow. The top teams spent Round 5 primarily on stability, not new strategies.

Master Strategy Reference for Prosperity 4

This section consolidates every major trading technique encountered across all three competitions. Study each concept before the competition begins so you can recognise which strategy applies when a new product is introduced.

1. Market Making (All Rounds, All Years)

Applicable to: Any product with a known or estimable fair value and a stable spread. Examples: PEARLS, AMETHYSTS, RAINFOREST_RESIN, KELP, individual components in basket rounds.

How it works: Post a bid at $(\text{fair_value} - \text{half_spread})$ and an ask at $(\text{fair_value} + \text{half_spread})$. When both fill, you earn the full spread. Profit = spread $\times$ fill_rate. Risk = adverse selection (someone fills your quote because they know more than you).

Key implementation details: (1) Fair value estimation — use Wall Mid (price of the largest standing order on each side) rather than arithmetic mid. (2) Skewing — if your position is long, widen your bid and tighten your ask to encourage selling; this passively manages inventory. (3) Soft/hard position limits — reduce quote size as you approach the limit, and place aggressive liquidation orders at the limit.

2. Statistical Arbitrage / Pairs Trading

Applicable to: Correlated product pairs or basket/component spreads. Examples: COCONUTS/PINA_COLADAS (P1), Gift Basket components (P2 R3), Picnic Baskets (P3 R2).

How it works: (1) Identify two products X and Y that are cointegrated. (2) Compute the spread $S = X - \text{beta} \times Y$ where beta is estimated by OLS regression on historical prices. (3) Compute Z-score: $z = (S - \text{rolling_mean}(S)) / \text{rolling_std}(S)$. (4) Enter when $|z| > \text{threshold}$ (typically 1.5–2.0), exit when z reverts toward 0. (5) For basket ETFs, beta is determined by the known composition ratio (no regression needed).

Python implementation skeleton: maintain a deque of recent spreads (maxlen ~50), compute mean and std, compute z-score, generate orders when threshold breached.

3. Options Pricing and Volatility Smile

Applicable to: COCONUT_COUPON (P2 R4), VOLCANIC_ROCK_VOUCHERS (P3 R3). Almost certainly relevant in P4.

How it works: (1) Estimate daily/per-tick volatility of the underlying from historical price data. (2) For each option at each strike, compute Black-Scholes theoretical price. (3) Invert the market price to find implied volatility (IV) via binary search. (4) Fit a quadratic function to the IV smile

across strikes. (5) Any option whose IV deviates significantly from the smile curve is mispriced — trade it. (6) Delta-hedge the resulting option book using the underlying.

Crucial parameters: annualisation factor for volatility (timestamps per day × days to expiry), correct $N(d1)$ computation using `scipy.stats.norm.cdf`, and sensible IV search bounds (0.01 to 3.0 is usually sufficient).

4. Conversion / Location Arbitrage

Applicable to: ORCHIDS (P2 R2), MAGNIFICENT_MACARONS (P3 R4). Likely in P4.

How it works: (1) Compute effective import cost = conversion ask + all fees. (2) Compute effective export revenue = conversion bid - all fees. (3) If local ask < import cost: import is expensive, buy locally. If local bid > export revenue: export is cheap, sell locally and hedge via conversion. (4) Execute the arbitrage: trade locally, convert in the same iteration to close the position flat. Never let a conversion position carry overnight.

Key nuance: the conversion can often be executed in the same iteration as the trade, meaning your position is zero at the end of each tick. This dramatically increases the profit rate compared to holding positions between ticks.

5. FX Arbitrage / Graph Cycles

Applicable to: All manual barter/exchange-chain problems across all three competitions. Likely in P4.

How it works: (1) Build a directed graph where nodes are goods/currencies and edge weights are exchange rates. (2) Use BFS/DFS to find the maximum-product cycle starting and ending at SeaShells within N steps. (3) Equivalently: take log of exchange rates, then find the maximum-weight cycle — convertible to a standard shortest-path problem. Implement once as a utility function and reuse every year.

6. External Signal / Fundamental Analysis

Applicable to: DIVING_GEAR/DOLPHIN_SIGHTINGS (P1 R4), MACARONS/SUNLIGHT_INDEX (P3 R4), News Trading (P3 R5).

How it works: (1) Plot the external signal against the price series to find the lag and correlation. (2) Build a simple threshold rule: if signal > X, take a long position; if signal < Y, go short. (3) The sunlight index in P3 created a clear regime shift — above/below a threshold completely changed the direction of the Macaron basis. Always visualise external data before coding.

7. Trader ID Front-Running

Applicable to: P2 R5, P3 R5. Very likely in P4 R5.

How it works: (1) Build a dictionary mapping `trader_id` to a list of (price, direction, subsequent_price_change) tuples. (2) For each bot, compute the average price impact at lag k for $k = 1, 2, \dots, 10$. (3) Bots with consistently positive impact after their buys (or negative after their sells) are front-runnable. (4) When you see a high-signal bot in the live feed, size into the expected direction immediately. Exit after k ticks or when impact exhausts.

8. Seasonal / Time-Series Patterns

Applicable to: BERRIES (P1 R4). May reappear.

How it works: (1) Compute the average mid-price at each timestamp index across multiple days/iterations. (2) If a clear cycle appears (price peaks at timestamp $T1$, troughs at $T2$), trade mechanically: build a long position before $T1$, exit after the peak; or build a short before $T2$, exit after the trough. (3) Phase risk: the season might shift by a few ticks — build your position gradually rather than all at once.

Tools, Preparation, and Execution for Prosperity 4

Pre-Competition Setup Checklist

The Frankfurt Hedgehogs (2nd place P2 and P3) emphasise that having the right tools prepared before the competition is the single biggest competitive advantage outside the strategies themselves. Here is what to prepare:

1. Install jmerle's open-source backtester (imc-prosperity-3-backtester on GitHub). Fork it and set it up locally before Day 1.
2. Build a visualisation dashboard that shows the live order book, mid-price history, spread history, and your position over time for each product. The Frankfurt Hedgehogs built a custom dashboard from scratch and credit it as essential for developing trading intuition.
3. Set up a logging infrastructure: your trader.py should log internal state (fair value estimate, current spread, position) at every tick. Use JSON logging for easy parsing.
4. Build utility functions in advance: (a) Wall Mid computation, (b) BFS for FX graph cycles, (c) Rolling Z-score computation, (d) Black-Scholes price and IV computation using scipy, (e) Position limit enforcement with soft/hard stops.
5. Study Prosperity 2 and 3 data (available on jmerle's GitHub) to understand the format: price_log, trade_log, order_depth structure.
6. Practice on the tutorial round: submit early, verify your logging and backtester work correctly, and get your baseline market-making algorithm running before the scored rounds begin.

Round-by-Round Strategy Roadmap

Round 1: Expect a stable product (pure market making) + one noisy product. Deploy Wall-Mid market making for the stable product. For the noisy product, implement AR(p) price prediction or modified market making. The manual problem is almost certainly an FX arbitrage or two-bid fish problem — have your BFS graph solver and Monte Carlo simulator ready.

Round 2: Expect an ETF / basket product. Have your stat-arb spread Z-score framework ready to deploy immediately. Or expect a conversion-arbitrage product (Orchid-style). Read the product description extremely carefully — the profitable mechanism is usually in the conversion mechanics, not the order book.

Round 3: Expect options or a volatility-related product. Have Black-Scholes pricing, IV inversion, volatility smile fitting, and delta hedging all pre-coded and tested. The manual problem may be a reserve-price game with a beauty-contest twist.

Round 4: Expect location arbitrage or an external signal product. Plot all available external data immediately. Look for regime changes. The manual problem is likely a suitcase/container EV problem.

Round 5: Trader ID data becomes available. Build front-running logic immediately. The manual problem may be a news trading / portfolio allocation task. Spend the rest of the round on stability and bug fixing across all 10–15 products.

AI-Assisted Strategy Development

Since Prosperity 4 is your first competition, using AI (including Claude) to help build and debug strategies is highly effective. Here is a suggested workflow:

For each new product introduced in a round, describe the product mechanics to Claude in detail (including position limits, any conversion rules, and example data). Ask: 'What trading strategies are typically used for this type of product? Write me a Python implementation.' Then backtest the resulting code using jmerle's backtester on historical Prosperity data to evaluate performance before submitting.

For the manual challenges, describe the problem mathematically and ask Claude to write a brute-force solver (for small problems) or derive the analytic optimum (for optimisation problems). Always verify the result with a Monte Carlo simulation.

For debugging: paste your trader.py and a failing log, and ask Claude to identify why positions are accumulating or why orders are not filling. The structured log format makes this very debuggable.

Common Mistakes to Avoid

- Ignoring position limits — always implement soft and hard liquidation.
 - Using arithmetic mid-price instead of Wall Mid for fair value in products with large standing orders.
 - Not reading the product description carefully — conversion mechanics, fees, and tariffs hide the true P&L opportunity.
 - Overfitting to backtest data — always hold out the final 20% of data for validation before deploying.
 - Picking the highest-multiplier container/suitcase without modelling crowding — always compute expected value after dilution.
 - Delta-hedging options infrequently — rebalance whenever net delta exceeds a small threshold (e.g., ± 5 units), not just once per round.
 - Trying to add new strategies in Round 5 — focus on stability, debugging, and extracting known alpha rather than introducing new risk.
 - Not building a backtester — teams without local backtesting capability have much longer iteration cycles and miss many optimisation opportunities.
-

Good luck in IMC Prosperity 4!

Study the strategies, build the tools, and trust the backtester.